

IPA ASSIGNMENT 18

Name – Nishant Nahar

Roll No – 241551078

1. Intensity Slicing in grayscale image

```
from PIL import Image
import numpy as np
image = Image.open("img_2.jpeg").convert("L")
img = np.array(image)
low = 100
high = 200
sliced = np.zeros_like(img)
sliced[(img >= low) & (img <= high)] = 255
output = Image.fromarray(sliced)
output.save("intensity_sliced.jpg")
image.show()
output.show()
print("Intensity slicing completed.")
print("Selected intensity range:", low, "to", high)
```

OUTPUT



2. Separate the color image into 3 types – RGB, CMYK and HIS

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_4.jpeg")
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
R = rgb[:, :, 0] G = rgb[:, :, 1] B = rgb[:, :, 2]
rgb_float = rgb.astype(float) / 255.0
R_norm = rgb_float[:, :, 0]
G_norm = rgb_float[:, :, 1]
B_norm = rgb_float[:, :, 2]
K = 1 - np.maximum.reduce([R_norm, G_norm, B_norm])
denominator = 1 - K
denominator[denominator == 0] = 1
C = (1 - R_norm - K) / denominator
M = (1 - G_norm - K) / denominator
Y = (1 - B_norm - K) / denominator
C = (C * 255).astype(np.uint8)
M = (M * 255).astype(np.uint8)
Y = (Y * 255).astype(np.uint8)
K_display = (K * 255).astype(np.uint8)
R = R_norm G = G_norm B = B_norm
I = (R + G + B) / 3
minimum = np.minimum.reduce([R, G, B])
S = 1 - (3 / (R + G + B + 1e-10)) * minimum
numerator = 0.5 * ((R - G) + (R - B))
denominator_h = np.sqrt((R - G) ** 2 + (R - B) * (G - B))
theta = np.arccos(np.clip(numerator / (denominator_h + 1e-10), -1, 1))
H = np.where(B <= G, theta, 2 * np.pi - theta)
H = H * 180 / np.pi
H_display = (H / 360 * 255).astype(np.uint8)
S_display = (S * 255).astype(np.uint8)
I_display = (I * 255).astype(np.uint8)
plt.figure(figsize=(12, 10)) plt.subplot(3, 3, 1)
plt.imshow(R * 255, cmap="gray") plt.title("R Component")
plt.axis("off") plt.subplot(3, 3, 2)
plt.imshow(G * 255, cmap="gray") plt.title("G Component")
plt.axis("off") plt.subplot(3, 3, 3) plt.imshow(B * 255, cmap="gray")
plt.title("B Component") plt.axis("off")
plt.subplot(3, 3, 4) plt.imshow(C, cmap="gray")
plt.title("C Component") plt.axis("off")
plt.subplot(3, 3, 5) plt.imshow(M, cmap="gray")
plt.title("M Component") plt.axis("off")
plt.subplot(3, 3, 6) plt.imshow(Y, cmap="gray") plt.title("Y Component")
plt.axis("off") plt.subplot(3, 3, 7)
```

```
plt.imshow(H_display, cmap="gray") plt.title("H Component")
plt.axis("off") plt.subplot(3, 3, 8)
plt.imshow(S_display, cmap="gray") plt.title("S Component")
plt.axis("off") plt.subplot(3, 3, 9)
plt.imshow(I_display, cmap="gray") plt.title("I Component")
plt.axis("off")
plt.tight_layout() plt.show()
```

OUTPUT

R Component



G Component



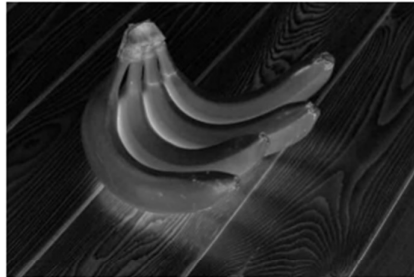
B Component



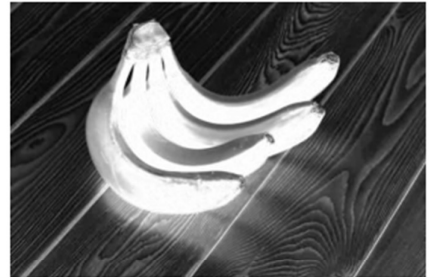
C Component



M Component



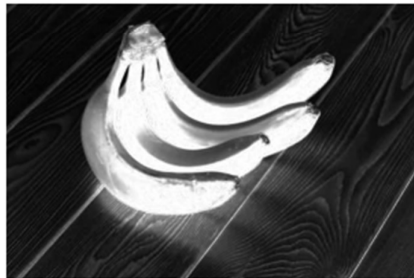
Y Component



H Component



S Component



I Component



3. Color Slicing of RGB Image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_4.jpeg")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lower_red1 = np.array([0, 100, 100])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 100, 100])
upper_red2 = np.array([180, 255, 255])
mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask = mask1 + mask2
result = cv2.bitwise_and(img, img, mask=mask)
plt.figure(figsize=(10, 4)) plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off") plt.subplot(1, 3, 2)
plt.imshow(mask, cmap="gray") plt.title("Color Slice")
plt.axis("off") plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Detected Red Objects")
plt.axis("off") plt.tight_layout() plt.show()
```

OUTPUT



4. Histogram Equalization of color image of RGB Model

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("img_5.jpeg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
R, G, B = cv2.split(img_rgb)
R_eq = cv2.equalizeHist(R) G_eq = cv2.equalizeHist(G)
B_eq = cv2.equalizeHist(B)
```

```

equalized = cv2.merge([R_eq, G_eq, B_eq])
plt.figure(figsize=(10, 5)) plt.subplot(1, 2, 1)
plt.imshow(img_rgb) plt.title("Original Color Image")
plt.axis("off") plt.subplot(1, 2, 2)
plt.imshow(equalized) plt.title("RGB Histogram Equalized")
plt.axis("off") plt.tight_layout() plt.show()

```

OUTPUT

Original Color Image



RGB Histogram Equalized



5. Comparing Histogram Equalization with both RGB and HIS Model

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
def rgb_to_hsi(img):
    rgb = img.astype(np.float32) / 255.0
    R = rgb[:, :, 0]    G = rgb[:, :, 1]    B = rgb[:, :, 2]
    I = (R + G + B) / 3.0
    minimum = np.minimum(np.minimum(R, G), B)
    S = 1 - (3 * minimum / (R + G + B + 1e-10))
    numerator = 0.5 * ((R - G) + (R - B))
    denominator = np.sqrt((R - G) ** 2 + (R - B) * (G - B))
    theta = np.arccos(np.clip(numerator / (denominator + 1e-10), -1, 1))
    H = np.where(B <= G, theta, 2 * np.pi - theta) H = H * 180 / np.pi
    return H, S, I
def hsi_to_rgb(H, S, I):
    H = H % 360
    R = np.zeros_like(H)    G = np.zeros_like(H)    B = np.zeros_like(H)
    sector1 = H < 120
    B[sector1] = I[sector1] * (1 - S[sector1])
    R[sector1] = I[sector1] * (1 + (S[sector1]
        * np.cos(np.radians(H[sector1]))
        / np.cos(np.radians(60 - H[sector1]))))
    G[sector1] = 3 * I[sector1] - (R[sector1] + B[sector1])

```



```

    sector2 = (H >= 120) & (H < 240)
    H2 = H[sector2] - 120
    R[sector2] = I[sector2] * (1 - S[sector2])
    G[sector2] = I[sector2] * (
        1 + (S[sector2] * np.cos(np.radians(H2)) /
np.cos(np.radians(60 - H2))))
    B[sector2] = 3 * I[sector2] - (R[sector2] + G[sector2])
    sector3 = H >= 240      H3 = H[sector3] - 240
    G[sector3] = I[sector3] * (1 - S[sector3])
    B[sector3] = I[sector3] * (
        1 + (S[sector3] * np.cos(np.radians(H3)) / np.cos(np.radians(60
- H3))))
    R[sector3] = 3 * I[sector3] - (G[sector3] + B[sector3])
    rgb = np.stack((R, G, B), axis=2)
    rgb = np.clip(rgb * 255, 0, 255)
    return rgb.astype(np.uint8)
img_bgr = cv2.imread("img_6.jpeg")
img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
R = img[:, :, 0] G = img[:, :, 1] B = img[:, :, 2]
R_eq = cv2.equalizeHist(R) G_eq = cv2.equalizeHist(G)
B_eq = cv2.equalizeHist(B)
rgb_equalized = cv2.merge([R_eq, G_eq, B_eq])
H, S, I = rgb_to_hsi(img)
I_8bit = np.clip(I * 255, 0, 255).astype(np.uint8)
I_eq_8bit = cv2.equalizeHist(I_8bit)
I_eq = I_eq_8bit.astype(np.float32) / 255.0
hsi_equalized = hsi_to_rgb(H, S, I_eq)
plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1) plt.imshow(img)
plt.title("Original Image")
plt.axis("off") plt.subplot(2, 3, 2)
plt.imshow(rgb_equalized)
plt.title("RGB Histogram Equalization")
plt.axis("off") plt.subplot(2, 3, 3)
plt.imshow(hsi_equalized)
plt.title("HSI Histogram Equalization")
plt.axis("off") plt.subplot(2, 3, 4)
plt.hist(I.ravel(), 256, range=(0, 1))
plt.title("Original Intensity Histogram")
plt.subplot(2, 3, 5) plt.hist(I_eq.ravel(), 256, range=(0, 1))
plt.title("HSI Equalized Intensity")
plt.subplot(2, 3, 6)
plt.hist(R_eq.ravel(), 256, range=(0, 255), alpha=0.4)
plt.hist(G_eq.ravel(), 256, range=(0, 255), alpha=0.4)
plt.hist(B_eq.ravel(), 256, range=(0, 255), alpha=0.4)
plt.title("Equalized RGB Channels") plt.tight_layout() plt.show()

```

OUTPUT



6. Smoothing of Color Image

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("img_8.jpeg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
smooth_3x3 = cv2.blur(img_rgb, (3, 3))
smooth_5x5 = cv2.blur(img_rgb, (5, 5))
plt.figure(figsize=(12, 4)) plt.subplot(1, 3, 1)
plt.imshow(img_rgb) plt.title("Original Image")
plt.axis("off") plt.subplot(1, 3, 2)
plt.imshow(smooth_3x3) plt.title("3x3 Smoothed Image")
plt.axis("off") plt.subplot(1, 3, 3)
plt.imshow(smooth_5x5) plt.title("5x5 Smoothed Image")
plt.axis("off") plt.tight_layout() plt.show()
cv2.imwrite("smoothed_3x3.jpg", cv2.cvtColor(smooth_3x3,
cv2.COLOR_RGB2BGR))
cv2.imwrite("smoothed_5x5.jpg", cv2.cvtColor(smooth_5x5,
cv2.COLOR_RGB2BGR))
print("Color image smoothing completed.")
```

OUTPUT

